Computer Architecture and Technology Area (ARCOS)

Universidad Carlos III de Madrid

OPERATING SYSTEMS

Practice 2. Programming an Uc3mshell

Bachelor's Degree in Computer Science and Engineering
Bachelor's Degree in Mathematics and Computing
Double Degree in Computer Science and Engineering and Business Administration

Academic year 2025/2026

Index

1. Laboratory statement.....2
1.1. Laboratory description.....2
2. Considerations.....3
2.1. Commands.....3
2.2. Sequence of commands.....3
2.3. Redirections.....4
2.4. Background.....5
2.5. Special case.....5
3. Command processing.....6
4. Command interpreter Uc3mshell development.....8
4.1. Internal command: exit.....8
4.2. Internal command: mycalc.....9
4.3. External command: mycp.....9
5. Supporting source code.....10
5.1. Provided Submission Checker.....11
6. Compilation and execution.....12
7. Submission.....12
7.1. Submission deadline.....12
7.2. Submission procedure.....12
7.3. Files to submit.....12
8. Rules.....15
9. Appendix.....16
9.1. man function.....16
9.2. Background and foreground modes.....16
9.3. External commands.....17
10. Bibliography.....18

1. Laboratory statement

This practice allows the student to become familiar with the process management services provided by POSIX. It is also intended that you know the inner workings of a UNIX/Linux shell. In short, a shell allows the user to communicate with the operating system kernel by executing commands, whether simple or chained.

For heavy process management, related POSIX system calls, such as *fork, wait,* or *exit*, will be used. For communication between processes, system calls to the *pipe, dup, close* or *signal* will also be used.

The student must design and code, in C language and in the UNIX/Linux operating system, a program that acts as a command interpreter or shell. The program must strictly follow the specifications and requirements contained in this document.

1.1. Laboratory description

Similar to the Linux bash command interpreter, the program to be developed is called "Uc3mshell" and must read a command file received as an input argument line by line and, using a command interpreter, execute them as if it were a native terminal.

The syntax to run this program is as follows:

```shell
./uc3mshell <input_file>
```

An example of an **input file** is shown below:

```shell
## Uc3mshell P2
ls -l
cat file | grep word

cat file > file2 &
#ls -l | sort > file1
ls -l | sort > file2
```

The header line (## Uc3mshell P2) is **mandatory**, so the Uc3mshell program must check its existence to know if the file is a valid script. If the input file does not contain the

header, the program must end with an error code -1 and return an error using the **perror function**.

Subsequently, the developed program must be able to execute, one by one, the commands listed in the file. The script can contain empty lines and comments starting with "#". In this case, the Uc3mshell should ignore these lines and move on to the next one. **The program must execute all commands, even if they end in error (failure of a command does not imply failure of the Uc3mshell).** The Uc3mshell must terminate:

1. When all the commands in the file have been completed.
2. When the system calls used in the program code (fork, exec, pipe, dup, etc.) have failed.
3. When the first line of the input file is different from "## Uc3mshell P2".

2. Considerations

1. A line can be a command with or without arguments (e.g., "ls", "ls -l", "ls -l directory").
2. A line can be a sequence of chained commands (via pipes), each with its respective flags and arguments (e.g., ls -l | wc).
3. Each command line can contain the following standardized characters:
   i. " **>** ": Output redirection.
   ii. " **<** ": Input redirection.
   iii. " **!>** ": Error redirection.
   iv. " **&** ": Indicates that the command will run in the background.
4. A line that begins with "#" is a comment. For example, "# Here the result is printed."

2.1. Commands

Therefore, **a command is understood** as a sequence of texts separated by spaces. The first text specifies the name of the command to be executed. The remaining ones are the arguments of the invoked command. For example, in the "ls -l." command, the command name is "ls", and the arguments are "-l" and ".".

The name of the command is passed as argument 0 (*man execvp*). Each command is executed as a direct child process of the Uc3mshell (*man 2 fork*). The value of a command is its termination state (*man 2 wait*). If the execution fails, the error is reported by the standard error output using the **perror function**.

2.2. Sequence of commands

A sequence of two or more commands is a series of commands separated by the character " | " (*man 2 pipe*). The output of the first command is automatically passed as input to the next one. The main program waits for the last command to finish before processing a new line. The result of the entire sequence will be the generated by the last command executed. For example, in "ls -l | wc -l" There are two commands: the output of the *ls* command is passed as input to the *wc* command to count the number of lines.

2.3. Redirections

A **redirection** indicates that the input or output of a command should come from or go to a file instead of the terminal. The input or output of a command or sequence of commands can be redirected by adding the following notation after it:

- command **< file** Use a **file** as **standard input,** opening it for reading. If the file does not exist, it ends with an error (*man 2 open*). **Input** redirections applies only to the **first** command:

```
Shell
command1 < file | command2 | command3 ...
```

- command **> file** Use a **file** as **standard output**. If the file does not exist, it is created. If it exists, it is truncated (*man 2 open / man creat*). **Output** redirections apply only to the **last** command:

```
Shell
command1 | command2 | ... | commandN > file
```

- command **!> file** Use a **file** as **standard error output**. If the file does not exist, it is created. If it exists, it is truncated (*man 2 open / man creat*). **Error** redirections apply only to the **last** command:

```
Shell
command1 | command2 | ... | commandN !> file
```

In case of any failure during redirections, it is notified by the standard error output (**perror**), the execution of that line is suspended, and the next line is executed.

It is possible to make multiple redirections in the same command or sequence of commands. For example, assuming "file.out" exists:

```shell
cat < file.out | tail -x10 > tail.out !> tail.err
```

1. The "cat" command will read the "file.out" file and send its output to the "tail" command.
2. The "tail" command will fail as "-x" is not a valid flag. The "tail.out" and "tail.err" files will be created. The first will be empty, while the second will contain the default error message: *tail: invalid option -- 'x'*.

2.4. Background

A command or sequence of commands ending in " **&** " implies asynchronous execution; that is, it runs in **the background**, allowing the main program to continue with the execution of the rest of the commands. The following should be considered:

1. The parent process must print the *pid* of the child process created and running in the background using the standard output (See Appendix section).
   Mandatory: the output must be (no extra comments):

```shell
printf("%d",pid);
```

2. The ending of the child processes must be managed correctly, as failure to do so will result in a penalty in the final grade.

2.5. Special case

A **special case** is the command "**echo**", in which the use of quotation marks must be properly controlled, if they exist. For example, the following commands generate exactly the same output:

```shell
echo "Hello world"
echo Hello world

# Expected output
```

```
Hello world
Hello world
```

The code provided **must be modified** to follow these **conditions**.

3. Command processing

For the development of this laboratory, it is necessary for students to process a command file. The name of this file is passed as an argument to the program. **They must read character by character until they find a line break and insert a '\0' at the end of the string of characters.** Once this step is implemented, each line can be passed as a parameter to the *procesar_linea* (process_line) function (included in the initial *uc3mshell.c* code)**,** which provides the basic functionality for the student to continue with the management of the process.

Variables to be used by students, included in the initial code:

- **char * argvv[]:** A structure equivalent to "argv" that stores the command line when executing a program. In this case, this structure will store the line corresponding to a command, along with its arguments.
- **char * filev[3]:** An array that indicates whether there is a redirection to or from a file. Each position corresponds to:
    - filev[0]: input redirection (file descriptor 0).
    - filev[1]: redirection of the standard output (file descriptor 1).
    - filev[2]: redirection of the error output (file descriptor 2).

  In case of redirections, the content of the position will be the pointer to the file name. Otherwise, the value of the position will be *NULL.*
- **int background:** If the command must run in the background, the value is 1; otherwise, it is 0.

The initial code (*uc3mshell.c*) includes, within the *procesar_linea* function, a code that displays the argvv, background and redirection (filev) contents on the screen. It is recommended to evaluate different cases once the input command file has been read and understood. It is then recommended that you modify that code to add the process execution functionality.

EXAMPLE: If the input file contains the following command:

```shell
ls -l | sort < fichero
```

The above arguments will have the arrangement shown in the following figure:

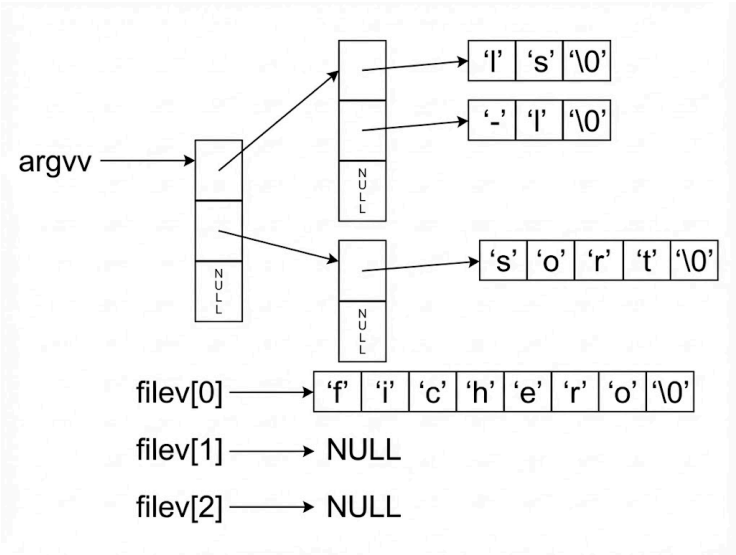

Figure 1: Data structure generated by Uc3mshell.

Note: To familiarize themselves with the structures in which commands, redirections, and background are returned, students are encouraged to compile and run the provided program to test different commands and command sequences in the input file. The goal is to clearly understand how to access each of the commands and ensure that they are decoded correctly. However, to do this, students must implement line-by-line reading of the input file and make use of the function provided: *int procesar_linea(char *linea);*

4. Command interpreter Uc3mshell development

To develop the Uc3mshell, students are recommended to follow a series of steps so that the Uc3mshell is built incrementally. In each step, a new functionality will be added on top of the previous one.

1. Reading the input file, character by character, recognizing command lines and adding '\0' as a delimiter.
2. Execution of simple commands such as ls, who, etc.
3. Execution of simple commands in "background" (&).
4. Execution of pipe-connected sequences of commands (|). The number of commands in a sequence will be limited to 3, for example: ls –l | sort | wc. If students decide to implement an unlimited sequence of pipes**, it will be considered for a complementary grade** (maximum grade 10).
5. Simple command and sequence of commands execution with standard input, standard output, and standard error redirections (<, >, !>), and executed in the background (&).
6. Execution of internal and external commands, which are detailed below.

4.1. Internal command: exit

The internal exit command must free up the memory used, if any, wait for the completion of the processes running in the background, delete zombie processes, print the message "**Goodbye <integer value>**", and finish the execution of the Uc3mshell by calling the system call exit(). It receives as an argument an integer that must be printed before finishing. For example:

```shell
echo "Hello world"
exit pepe
exit
exit 3

# Expected output
Hello world
[ERROR] The exit code must be an integer
[ERROR] Missing exit code
Goodbye 3
```

4.2. Internal command: mycalc

The Uc3mshell must support the internal mycalc command, whose syntax is:

mycalc <operation 1> < + | - | x | / > <operation 2>

To do this, the student must:

- Verify that the syntax is correct.
- Execute the chosen operation: + (addition), - (subtraction), x (multiplication), / (division).
- Display the results on the screen.

For example:

```shell
echo "Hello world"
mycalc 1 + 1

# Expected output
Hello world
Operation: 1 + 1 = 2
```

As it is an internal command, a header file (*mycalc.h*) must be created where all the functions necessary to invoke the mycalc command will be declared, and their behavior will be defined in an implementation file (*mycalc.c*). The source file of the Uc3mshell (*uc3mshell.c*) must include the created header file (*#include "mycalc.h"*).

4.3. External command: mycp

mycp is an external program that must be developed and compiled by the student independently of the Uc3mshell, that is, an executable must be generated when compiling it. The behavior of this program is similar to the basic "**cp**" command implemented in the Linux shell: it creates a copy of a file. The name of the copy is given as a parameter of the program.

The Uc3mshell must be able to execute *mycp,* whose syntax is:

mycp <source file> <destination file>

To do this, you must:

- Verify that the syntax is correct.
- Check that the source file exists and read it.
- Create a new file with the new name passed as a parameter.
- If the destination file exists, it must be overwritten.
- Close the open files.

- *Note*: verify that the new file exists.

For example:

```shell
echo "Hello world"
mycp file1.txt renamed.txt

# Expected output
Hello world
```

5. Supporting source code

To facilitate carrying out this laboratory practice, the ***p2_uc3mshell.zip* file** is available. It contains supporting source code and can be downloaded from Aula Global. To extract its contents, the following command must be executed:

```shell
unzip p2_uc3mshell.zip
```

When you extract its contents, the **p2_uc3mshell/** directory is created, where the practice is to be developed. The following files are included within this directory:

Makefile

Source file for the make tool. **It should NOT be modified**. With it, automatic compilation is achieved only for the source files that are modified.

uc3mshell.c

C source file. This file is the one that must be modified for the development of the command processor.

mycp.c

C source file. This file is the one that must be modified for the development of mycp.

mycalc.c

C source file. This file is the one that must be modified for the definition of mycalc functions.

mycalc.h

C header file. This file is the one that must be modified for the declaration of mycalc functions.

autores.txt

Must be modified. Text file (.txt) containing the authors of the laboratory (one per line). Format (without accents):
NIA1,Surname(s),Name(s)
NIA2,Surname(s),Name(s)
NIA3,Surname(s),Name(s)

```Textproto
Valid example:
100499455,Surname1 Surname2,Name
100499455,Surname1 Surname2,Name1 Name2
```

```Textproto
Invalid example:
,Surname1 Surname2,Name1 (Not include NIA)
(Whitespace)
Surname1 Surname2,Compound name,100499455
499455,Surname1 Surname2,Name (Incomplete NIA)
```

5.1. Provided Submission Checker

Students are provided with a Python script (version 3) called "**checker_ssoo_p2.py**" that verifies that the format of the laboratory assignment is correct. The script checks that the assignment follows the naming specifications, is properly compressed, includes everything specified, and **compiles correctly**.

The checker must be run with the following command:

```Shell
python3 checker_ssoo_p2.py <deliverable.zip>
```

Where **deliverable.zip** is the file to be submitted via Aula Global (see the Submission section).

For example:

```Shell
python3 checker_ssoo_p2.py ssoo_p2_100254896_100047014.zip
```

The checker prints messages on the screen showing if the format is correct or not.

6. Compilation and execution

To compile and run the lab, you must use the "make" command. The initial rules are already included in the provided Makefile.

Example **compilation**:

```shell
make clean
make uc3mshell
make mycp
```

Example of **execution**:

```shell
./uc3mshell <input_file>
```

7. Submission

The practice will be carried out in teams of up to 3 members from the **same small group**.

7.1. Submission deadline

The deadline for submitting the laboratory practice assignment to AULA GLOBAL is **Friday, April 10, 2026, at 11:55 p.m.**

7.2. Submission procedure

The laboratory practice assignments must be submitted electronically. Links will be provided on AULA GLOBAL through which students can submit their assignments. Specifically, **one link will be provided for submitting the laboratory practice assignment code, and another TURNITIN link will be provided for submitting the laboratory practice assignment report**.

7.3. Files to submit

A compressed file in zip format must be submitted in the link for submitting the laboratory practice assignment code with the name:

ssoo_p2_NIA1_NIA2_NIA3.zip

where NIA1, NIA2, and NIA3 are the NIAs of the members of the group. The file must contain:

- uc3mshell.c
- Makefile
- mycp.c
- mycalc.c
- mycalc.h
- autores.txt: **Text file (.txt) containing the authors of the laboratory (one per line). Format (without accents) as indicated above.**

In the TURNITIN link, the report of the practice must be submitted in PDF format with the name:

ssoo_p2_NIA1_NIA2_NIA3.pdf

The report must contain at least the following sections:

- **Cover page:** with the full names of the authors, NIAs and email addresses.
- **Index.**
- **Description of the code** detailing the main functions implemented. DO NOT include source code of practice in this section. Any code will be automatically ignored.
- **Battery of tests** used and results obtained. Higher scores will be given to advanced tests, extreme cases and, in general, to those tests that guarantee the correct functioning of the practice in all cases. It should be considered:
    - That a program compiles correctly and without warnings is not a guarantee that it will work correctly.
    - Avoid duplicate tests that evaluate the same program flows. The grade in this section is not measured according to the number of tests, but the degree of coverage of them. It is better to have a few tests that evaluate different cases than many tests that always evaluate the same case.
- **Conclusions**, problems encountered, how they have been solved and personal opinions.

The following aspects related to the **presentation of the assignment** will also **be** graded:

- It must contain a cover page with the authors of the practice and their NIAs.

- It must contain a table of contents.
- The memory must have page numbers on all pages (except the cover page).
- The text of the report must be justified.

The length of the report must not exceed 15 pages (cover page and index included). It is essential to pass the report in order to pass the laboratory practice, so you should not neglect the quality of the report.

Note: It is possible to submit the practice code as many times as you want within the delivery period, with the last delivery being the final version.
Considerations for using AI: If AI tools are used and solutions from multiple groups are identical, it will be considered plagiarism. Moreover, please note that this course includes a midterm and a final exam, both of which require students to write programs in C. Students without sufficient proficiency in C will not be able to successfully complete these exams.
THE REPORT CAN ONLY BE SUBMITTED ONCE THROUGH TURNITIN

8. Rules

1. Practices that do not compile or that do not conform to the functionality and requirements set out will obtain a grade of 0.

2. Programs that use library functions (**fopen, fread, fwrite, fclose, etc.**) or similar, instead of system calls, will get a grade of 0. The use of statements or functions such as **goto** is also not permitted.

3. Special attention will be paid to detecting functionalities copied between two practices. In case of finding common implementations in two practices, the students involved (copyers and copiers) will lose the grades obtained by continuous assessment.

4. Programs should compile without **warnings**.

5. The programs must run under a Linux system. Developing the practice for other systems (such as Windows or macOS) is not allowed. In addition, to ensure the correct functioning of the practice, its compilation and execution must be checked on the Guernika server (https://www.lab.inf.uc3m.es/servicios/aulas-virtuales-del-laboratorio/). If the code presented does not compile or does not work on top of this platform, the implementation will not be considered correct.

6. An uncommented program will get a **very low grade**.

7. The laboratory practice assignment will be delivered through Aula Global, as detailed in the Submission section of this document. Delivery via email without prior authorization is not permitted.

8. The format of the input and output indicated in each program to be implemented must be respected at all times.

9. Error control must be carried out in each of the programs, **more exhaustive than explicitly requested in each section**.

Submitted programs that do not follow these guidelines will not be considered approved.

9. Appendix

9.1. man function

man is the pager of the system manual, i.e. it allows you to search for information about a program, a utility or a function. See the following example:

man 2 fork

Pages used as arguments when executing man are usually the names of programs, utilities, or functions. Typically, the search is carried out in all available man sections in a predetermined order, and only the first page found is presented, even if that page is in multiple sections.

To exit the displayed page, simply press the 'q' key.

A man page has several parts. These are labeled NAME, SYNOPSIS, DESCRIPTION, OPTIONS, FILES, SEE ALSO, BUGS, and AUTHOR. The SYNOPSIS label lists the libraries (identified by the #include directive) that must be included in the user's C program in order to be able to use the corresponding functions.

The most common ways to use *man* are as follows:

- **man item section:** Presents the item page available in the manual section.
- **man -a element:** Presents, sequentially, all the pages of the elements available in the manual. Between pages, you can decide to skip to the next page or exit the pager completely.
- **man -k keyword:** Search for the keyword among the short descriptions and the man pages and present all the ones that match.

9.2. Background and foreground modes

When a sequence of commands is executed in the background, the *pid* that is printed is that of the process that executes the last command in the sequence.

When a simple command is executed in the background, the *pid* that is printed is that of the process that executes this command.

With the background operation, it is possible for the uc3mshell process to display the prompt mixed with the output of the child process. This is correct.

After running a command in the foreground, the uc3mshell cannot have zombie processes from previous commands run in the background.

9.3. External commands

External commands (mycp) are executed like any other program, executed on top of a process. Therefore:

- They are part of the sequences of commands.
- They have file redirections.
- They run in the background.

10. Bibliography

- El lenguaje de programación C: diseño e implementación de programas. Félix García, Jesús Carretero, Javier Fernández y Alejandro Calderón. Prentice-Hall, 2002.
- The UNIX System S.R. Bourne Addison-Wesley, 1983.
- Advanced UNIX Programming M.J. Rochkind Prentice-Hall, 1985.
- Sistemas Operativos: Una visión aplicada. Jesús Carretero, Félix García, Pedro de Miguel y Fernando Pérez. McGraw-Hill, 2001.
- Unix man pages (man function)